ARK Selection Task Perception Team: Noise Filtering

Aashi Asmita

***Abstract*—**
In this task, we are primarily working on filtering out noise from the given images. The first image consists of Salt Noise which is removed by identifying the boundary of the primary object in the image generated by Binary Thresholding and then removing all extra specks. The second image consists of Gaussian Noise which is reduced by using a combination of Gaussian Blur and Bilateral Filter. Noise filtering plays a key role in enhancing image quality to improve performance in computer vision tasks like edge detection and object recognition, which plays a vital role in Aerial Robotics.

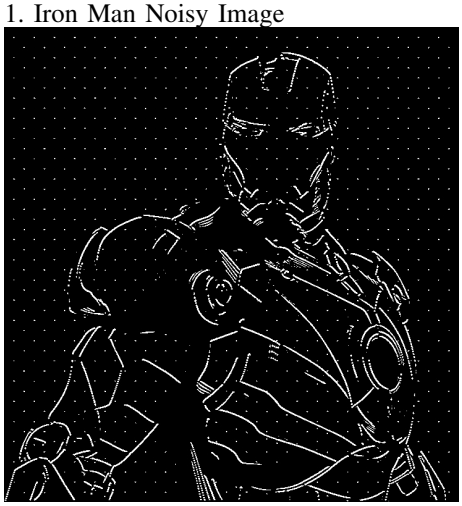
I. INTRODUCTION

For the Noise filtering task, we were provided with 2 corrupted images, an image of Iron Man which was heavily degraded by Salt Noise and a scenery which was degraded by Gaussian Noise. For the first image, I have first applied Binary Thresholding, then used the output image to detect the edges of the primary object. A minimum area is initialized and all specks smaller than the area are removed. I also tried using a combination of Median Blur and Morphological opening, but that resulted in noticeable damage in the original figure as well. For the second image I am using a combination of Gaussian blur and a bilateral filter to reduce the noise in the provided image. Although it did not produce a completely clean image, using other methods like Median Blur, NLM, etc produced even worse results.

II. PROBLEM STATEMENT

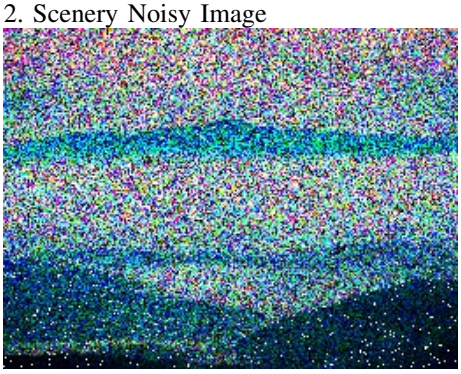
In this particular task, we were provided 2 corrupt images for noise filtering. They are as follows:

1. Iron Man Noisy Image



The above image was provided to us, which contains an outline of Iron man which is degraded by Salt Noise(white spots which are distributed all over the provided image). Our goal was to find the best suited method and parameters to filter out the noise and to restore the clean line drawing as much as possible.

2. Scenery Noisy Image



The above image provided to us comprises of a scenery, which is heavily degraded due to Gaussian Noise applied all over the image. Our objective was to filter out the noise and generate a clean image of the scenery by applying methods suitable to deal with such type of noise.

III. RELATED WORK

Apart from the standard methods that are used for image denoising such as Gaussian Blur, Median Blur, Bilateral Filter, enhanced denoising methods such as NLMs(Non-Local Means) and CNNs(Convolutional Neural Networks) can also be used. NLM operates through applying the formula that divides the sum of all pixels' values in the image by their corresponding weights, if the pixels are similar to the referenced pixel. CNNs employ multiple layers of the convolutional filter, which is used to reconstruct image features and, therefore, minimize noises.

IV. INITIAL ATTEMPTS

For the Iron Man image, I first tried to apply Median Blur, which replaces the value of each pixel with the median value of its neighbors. This preserves the sharpness of the edge. This was followed by the Morphological opening, which removes small, bright noise or objects (speckles) and breaks thin connections between larger objects while preserving their overall shape and size. However, this led to significant loss of edges of the primary subject of the image and was thus discarded.

For the scenery image, my first instinct was to apply Bilateral filter. A bilateral filter is a non-linear, edge-preserving, and noise-reducing smoothing filter used in image processing to blur images while keeping sharp edges intact. Unlike Gaussian filters, it considers both spatial distance (pixel proximity) and intensity difference (color similarity) to calculate

weights, preventing edges from being smoothed out. This made it seem like a good choice for the given problem. However, it did not produce the desired result and thus was dropped.

V. Final Approach

The objective of this implementation is to remove noise from two corrupted images using image processing techniques in OpenCV. The first image contains strong salt noise affecting a line drawing, while the second image contains natural image noise. Two different pipelines are implemented to handle these cases effectively.

1. Noise Removal Using Connected Components (Iron Man Image)

The first part of the code focuses on cleaning the Iron Man noisy image, which mainly contains salt noise. Salt noise appears as isolated white pixels scattered across the image. Since these pixels usually form very small clusters, connected component analysis can be used to remove them.

First, the image is loaded in grayscale using **cv2.imread()**. Working in grayscale simplifies the processing because only intensity values need to be analyzed.

Next, the image is converted into a binary image using thresholding. Thresholding separates the image into two intensity levels: background (0) and foreground (255). This makes it easier to detect connected structures in the image.

The next step performs connected component analysis:

The **cv2.connectedComponentWithStats()** function scans the binary image and identifies all groups of connected pixels. Each group is treated as a component and statistics such as its area are computed. The parameter connectivity=8 allows diagonal neighbors to be considered connected, which helps detect more natural shapes.

A blank image clean is then created to store the filtered result. A minimum area threshold is defined. Small connected components are assumed to be noise. The code iterates through all detected components and checks their area. The background component (index 0) is ignored. If the component's area is larger than the threshold, it is preserved; otherwise it is removed. This effectively removes small noisy pixel clusters while preserving the larger structures of the drawing.

Finally, the cleaned image is resized for visualization and displayed.

This approach works well because noise pixels usually form very small connected components, whereas meaningful parts of the drawing form larger structures.

2. Noise Reduction for Natural Images

The second part of the code processes a different noisy image that contains general image noise rather than isolated salt noise.

The image is first loaded and converted from BGR to RGB format for correct color representation.

The first filtering step applies Gaussian Blur.

newimg=cv2.GaussianBlur(image,(3,3),0)

Gaussian blur smooths the image by averaging neighboring pixel values using a Gaussian kernel. This reduces high-frequency noise while maintaining the overall structure of the image.

After Gaussian smoothing, a bilateral filter is applied.

newimg=cv2.bilateralFilter(newimg, d=21, sigmaColor=200, sigmaSpace=150)

Unlike Gaussian blur, bilateral filtering preserves edges while smoothing flat regions. It works by considering both the spatial distance between pixels and the intensity difference between them. This makes it very effective for noise reduction without blurring important boundaries.

The parameters used are:

d = 21: diameter of the pixel neighborhood

sigmaColor = 200: how much color difference is tolerated

sigmaSpace = 150: how far neighboring pixels influence the filtering

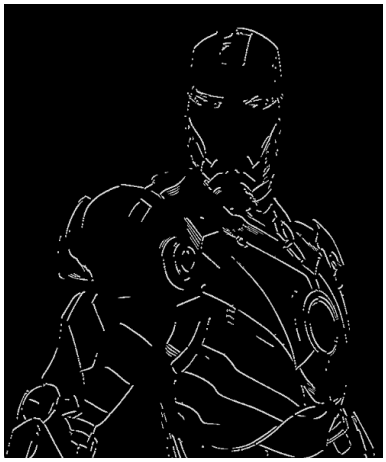
After filtering, the image is converted back to the correct color format and displayed.

VI. Results and Observation

The initial algorithm for the Iron man problem yielded the following result:

As we can see, some of the edges are also lost in this process. Thus, this method was deemed unsuitable.

Using the final algorithm, we obtain the following final result:



The initial algorithm for the Scenery problem yielded the following result:

As we can see, this algorithm was unable to denoise the image to an optimum extent. Thus, this method was deemed unsuitable.

Using the final algorithm, we obtain the following final result:

CONCLUSION

This code demonstrates two different noise removal strategies based on the type of noise present in the image. For images with salt noise, connected component analysis effectively removes small noisy regions. For natural images with general noise, a combination of Gaussian blur and bilateral filtering provides smooth results while preserving edges. Together, these methods produce cleaner images without significantly degrading important visual structures.

Noise filtering plays a key role in enhancing image quality to improve performance in computer vision tasks like edge detection and object recognition, which plays a vital role in Aerial Robotics.

REFERENCES

[1] OpenCV. (2024). OpenCV-Python Tutorials. Available at: https://docs.opencv.org/

[2] https://www.geeksforgeeks.org/computer-vision/what-are-the-different-image-denoising-techniques-in-computer-vision/